

PROJECT DOCUMENTATION: Personal Expense Tracker

1. Introduction:

- Project Title: Personal Expense Tracker
- Team Members:

Names	Roles
Vikesh Mehta	Frontend Development and Database
Ishan Peshkar	Backend Development and Database
Venus Agrawal	Database and Documentation
Vedant Wadhwa	Database and Documentation Work

2. Project Overview:

- Purpose: To provide users with a comprehensive and user-friendly web application for managing personal finances. The primary goal is to enable users to easily track expenses, monitor spending against budgets, analyze financial patterns, and gain better control over their financial health.
- Features:
 - User Authentication: Secure user registration and login using JWT and password hashing.
 - Expense Tracking: CRUD operations (Create, Read, Update, Delete) for expenses.
 - Categorization: Assigning expenses to predefined categories (including personal, family, student, basic business types).
 - Budget Management: Setting, viewing, updating, and deleting budgets per category for various periods (Monthly, Yearly, etc.). Visual progress tracking against budgets.
 - Reporting: Generating reports on spending by category (doughnut chart) and budget vs. actual spending (table) based on user-selected time periods.
 - Notifications: Backend alerts for nearing/exceeding budget limits, displayed via a notification bell in the frontend (polling mechanism).
 - INR Currency: Display and handling of monetary values in Indian Rupees (₹).
 - Dynamic UI: Animated interface elements and transitions using Framer Motion.
 - Responsive Design: Basic responsive layout considerations.

3. Architecture:

- Frontend:
 - Built with React.js (using functional components and Hooks like useState, useEffect, useContext, useCallback).
 - React Router DOM (v6) for client-side routing and navigation (/login, /register, /dashboard, /budgets, /reports).
 - Context API (useAuth hook) for global state management (authentication status, user info, notifications).
 - Axios for making asynchronous HTTP requests to the backend API.
 - Chart.js & react-chartjs-2 for data visualization in reports.
 - Framer Motion for UI animations and transitions.
 - React Icons for iconography.
 - Styling managed with CSS3 and CSS Variables for theming and consistency.
- Backend:
 - Built with Node.js and the Express.js web framework.
 - Follows a structure similar to MVC (Models, Controllers, Routes).
 - Mongoose ODM used for modeling application data and interacting with MongoDB.
 - RESTful API design for handling frontend requests.
 - Middleware used for authentication (protect middleware verifying JWT) and request parsing (express.json()).
 - JWT (jsonwebtoken) for generating and verifying authentication tokens.
 - bcryptjs for securely hashing user passwords before storing them.
 - dotenv for managing environment variables.
 - cors middleware to enable Cross-Origin Resource Sharing.
- Database:
 - MongoDB (NoSQL database, using MongoDB Atlas cloud hosting or local instance).
 - Schemas: Defined using Mongoose for:
 - User: Stores user credentials (name, email, hashed password). Email is unique.
 - Expense: Stores expense details (user reference, description, amount, category, date, notes, vendor, paymentMethod, project, isReimbursable, receiptUrl). Linked to a User via ObjectId.
 - Budget: Stores budget details (user reference, category, amount, period, startDate, endDate). Linked to a User via ObjectId. Indexed for uniqueness on user/category/startDate/period.
 - Notification: Stores user-specific notifications (user reference, message, type, isRead).
 - Interactions: Mongoose methods (.find(), .findById(), .create(), .findByIdAndUpdate(), .findByIdAndDelete(), .aggregate()) are used within

controllers to interact with the database based on API requests.
Aggregation framework used for generating report data.

4. Setup Instructions:

- Prerequisites:
 - Node.js (v16 or later recommended) & npm (comes with Node.js)
 - MongoDB (Cloud Atlas account or local installation)
 - Git (Optional, for cloning)
- Installation:
 1. Clone: `git clone <your-repository-url> expense-tracker-app` or download and extract the project ZIP.
 2. Navigate: `cd expense-tracker-app`
 3. Backend Setup:
 - `cd server`
 - `npm install`
 - Create a `.env` file in the server directory.
 - Add `MONGO_URI` (your MongoDB connection string) and `JWT_SECRET` (a strong random string) to the `.env` file. Optionally add `PORT=5001`.
- Frontend Setup:
 - `cd ../client` (from the root `expense-tracker-app` folder)
 - `npm install`

5. Folder Structure:

- Client (client/src/):
 - components/: Reusable UI parts (auth forms, expense items, budget forms, charts, layout elements like Navbar, NotificationBell, common elements like Spinner). Organized into subfolders (e.g., auth, expenses, budgets, reports, layout, common).
 - contexts/: React Context files for global state (e.g., AuthContext.js).
 - hooks/: Custom React hooks (e.g., useAuth.js).
 - pages/: Top-level components representing application pages/views (e.g., LoginPage.js, DashboardPage.js, BudgetPage.js).
 - services/: Functions dedicated to making API calls to the backend (e.g., authService.js, expenseService.js).
 - utils/: Helper functions (e.g., formatters.js).
 - App.js: Main application component, sets up routing.
 - index.js: Application entry point, renders App component.
 - index.css: Global styles and CSS variable definitions.

- **Server (server/):**
 - config/: Database connection logic (db.js).
 - controllers/: Functions handling the logic for each route (request handling, database interaction, response sending).
 - middleware/: Custom middleware functions (e.g., authMiddleware.js for JWT protection).
 - models/: Mongoose schema definitions for database collections (e.g., User.js, Expense.js).
 - routes/: Express router files defining API endpoints and linking them to controller functions.
 - .env: Environment variables (ignored by Git via .gitignore).
 - server.js: Main backend application entry point, sets up Express app, middleware, and mounts routers.

6. Running the Application:

- **Backend:**
 - Navigate to the server directory in your terminal.
 - Run: `npm run server` (uses nodemon for auto-restart during development) or `npm start` (uses node).
 - Server typically runs on `http://localhost:5001`.
- **Frontend:**
 - Navigate to the client directory in a separate terminal.
 - Run: `npm start`
 - Application opens automatically in your browser, typically at `http://localhost:3000`.

7. API Documentation:

Method	Endpoint	Protected	Parameters / Body	Success Response (200)	Error Response (400/500)	Description
POST	/api/auth/register	No	Body: { name, email, password }	201 { success: true, token, user }	400 { success: false, message } (Validation/Exists)	Register a new user.
POST	/api/auth/login	No	Body: { email, password }	200 { success: true, token, user }	400/401 { success: false, message } (Validation/Creds)	Log in a user.
GET	/api/auth/me	Yes	-	200 { success: true, data: user }	401 { success: false, message } (No/Invalid Token)	Get current logged-in user details.
GET	/api/expenses	Yes	-	200 { success: true, count, data: [expenses] }	401/500 { success: false, message }	Get all expenses for the logged-in user.
POST	/api/expenses	Yes	Body: { description, amount, category, date, ... }	201 { success: true, data: newExpense }	400/401/500 { success: false, message }	Add a new expense.
PUT	/api/expenses/id	Yes	Param: id, Body: { fields_to_update }	200 { success: true, data: updatedExpense }	400/401/404/500 { success: false, message }	Update an existing expense.
DELETE	/api/expenses/id	Yes	Param: id	200 { success: true, data: {}, message }	401/404/500 { success: false, message }	Delete an expense.
GET	/api/budgets	Yes	-	200 { success: true, count, data: [budgets] }	401/500 { success: false, message }	Get all budgets for the user (with spending).
POST	/api/budgets	Yes	Body: { category, amount, period, startDate, ... }	201 { success: true, data: newBudget }	400/401/500 { success: false, message }	Add a new budget.
PUT	/api/budgets/id	Yes	Param: id, Body: { fields_to_update }	200 { success: true, data: updatedBudget }	400/401/404/500 { success: false, message }	Update an existing budget.
DELETE	/api/budgets/id	Yes	Param: id	200 { success: true, data: {}, message }	401/404/500 { success: false, message }	Delete a budget.
GET	/api/reports/category-summary	Yes	Query: period (e.g., 'Monthly'), date (optional)	200 { success: true, period, ... data: [...] }	400/401/500 { success: false, message }	Get spending grouped by category.
GET	/api/reports/budget-status	Yes	Query: period (e.g., 'Monthly'), date (optional)	200 { success: true, period, ... data: [...] }	400/401/500 { success: false, message }	Get budget vs. actual spending data.
GET	/api/notifications/unread	Yes	-	200 { success: true, count, totalUnread, data }	401/500 { success: false, message }	Get user's unread notifications.
PUT	/api/notifications/id/read	Yes	Param: id	200 { success: true, data: notification }	401/404/500 { success: false, message }	Mark a single notification as read.
PUT	/api/notifications/readall	Yes	-	200 { success: true, message }	401/500 { success: false, message }	Mark all user's notifications as read.

8. Authentication:

- Mechanism: Authentication is handled using JSON Web Tokens (JWT).
- Process:
 1. User registers or logs in with email and password.
 2. Password is hashed using bcryptjs before comparison or storage.
 3. Upon successful authentication, the backend generates a JWT containing the user's ID.
 4. The JWT is signed using a secret key (JWT_SECRET environment variable).
 5. The token is sent back to the client.
 6. The client stores the token (e.g., in localStorage).
 7. For subsequent requests to protected API endpoints, the client includes the token in the Authorization header as a Bearer token (Authorization: Bearer <token>).
- Authorization:
 - The backend uses custom middleware (authMiddleware.js -> protect) applied to protected routes.
 - This middleware extracts the token from the Authorization header.
 - It verifies the token using the JWT_SECRET.
 - If valid, it fetches the user details (excluding password) from the database using the ID in the token payload.
 - The user object is attached to the req object (req.user), making it available to subsequent controller functions.
 - If the token is missing, invalid, or the user doesn't exist, the middleware sends a 401 Unauthorized response, preventing access to the route.
 - Controllers further ensure users can only access/modify their own data (e.g., expenses, budgets) by comparing req.user.id with the user field stored in the data documents.

9. User Interface:

- The user interface is built as a Single Page Application (SPA) using React.
- Key UI components include:
 - Login/Register: Animated forms with input validation and icons within a dedicated layout featuring project branding.
 - Navbar: Sticky navigation bar providing access to main sections (Dashboard, Budgets, Reports), displaying user info, and housing the notification bell. Features hover and active link effects.
 - Dashboard: Main view displaying a summary (total expenses), an animated form (collapsible) for adding/editing expenses, and a list of recent expenses.
 - Expense List/Item: Dynamically rendered list of expenses with animations, showing details (date, description, category, amount in INR, project), and providing Edit/Delete actions.

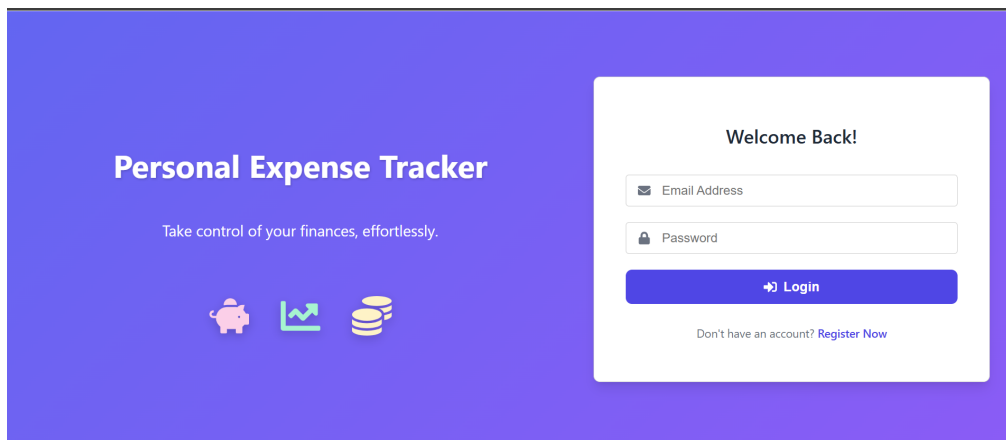
- Budget Page: Contains components for adding/editing budgets (BudgetForm) and displaying existing budgets (BudgetList, BudgetItem) with progress bars indicating spending status.
- Reports Page: Features filters for selecting report periods and displays data using a doughnut chart (CategoryChart) and a formatted table (BudgetStatusTable).
- UX Elements: Uses Framer Motion for smooth page transitions and component animations (entry, exit, hover effects). Utilizes React Icons for clear visual cues. State management via Context ensures UI updates reactively to data changes (e.g., authentication status, notifications).

10. Testing:

- Strategy: Primarily Manual Testing was employed due to the project scope and timeline.
- Process:
 - Functionality Testing: Manually testing all features - registration, login, logout, adding/editing/deleting expenses and budgets, filtering reports, checking calculations.
 - UI/UX Testing: Verifying layout, responsiveness (basic checks on different screen sizes), animations, and overall user experience across different browsers (e.g., Chrome, Firefox).
 - API Testing (Manual): Using tools like Postman or Insomnia during development to test backend endpoints independently for correct responses, error handling, and authentication protection.
 - Error Handling: Intentionally providing invalid inputs or attempting unauthorized actions to check if appropriate error messages and responses are handled.
- Tools: Browser Developer Tools (Console, Network Tab, Elements Tab), Postman/Insomnia (for API checks).

11. Screenshots or Demo:

Login Page:



Registration Page:

Personal Expense Tracker

Take control of your finances, effortlessly.



Welcome Back!

Login

Don't have an account? [Register Now](#)

Dashboard:

Expense Tracker

DashboardBudgetsReports

Hi, John Doe!

Dashboard

Welcome, John Doe! Logout

Add New Expense

Total Recorded Expenses: ₹500.00

Your Expenses

7 Apr 2025

I have went for buying groceries for my home. (General Store)

Groceries

₹500.00

Expense Form (Expanded):

Expense Tracker

BudgetsReports

Add New Expense

Description *

Amount (INR) *

e.g., 1500.50

Category *

Date *

Groceries

12 - 04 - 2025

Vendor

Payment Method

Business Details (Optional)

Project/Client

Receipt URL

Upload Receipt

Link to uploaded receipt

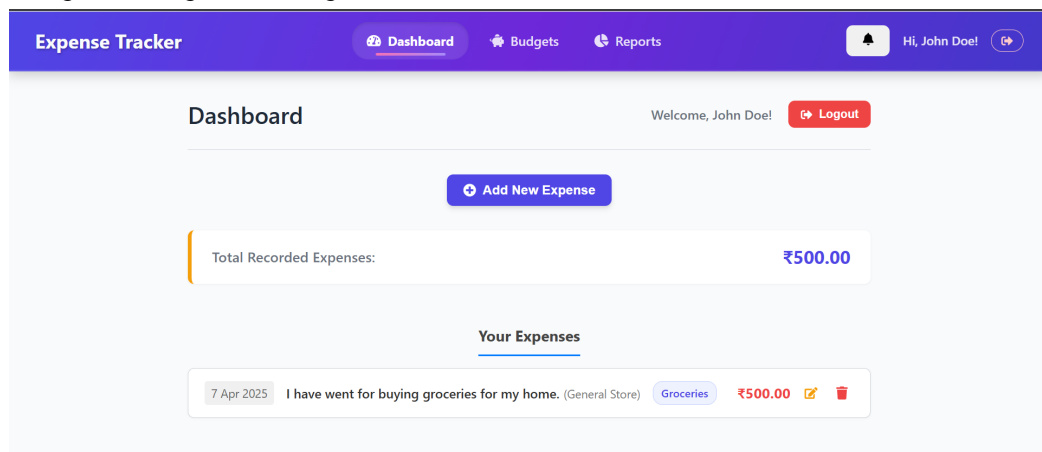
Choose File No file chosen

(File upload not functional in this version)

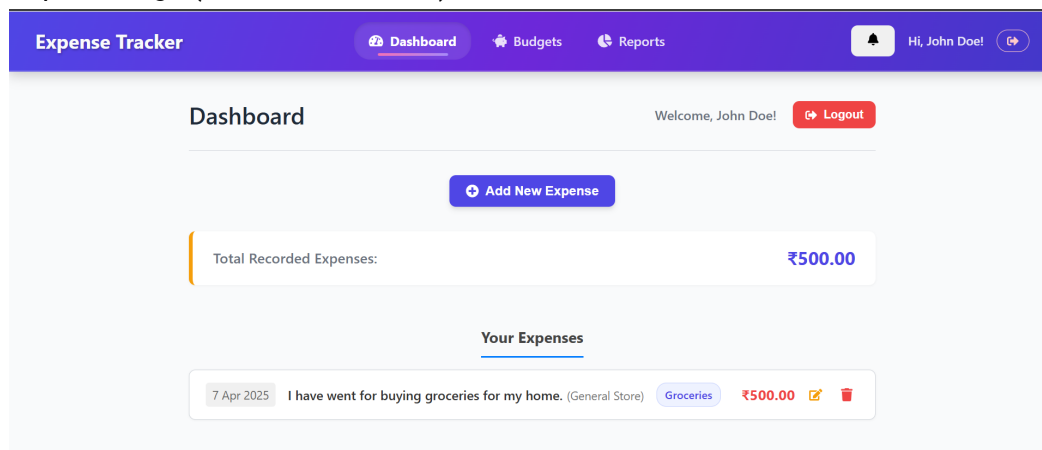
☐

Is this expense reimbursable?

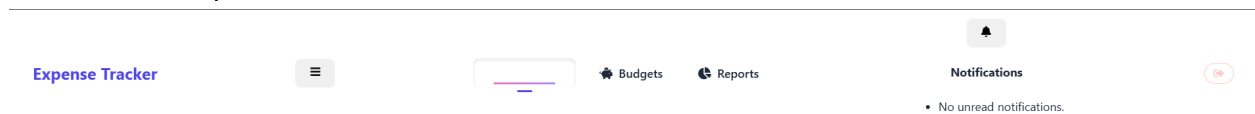
Budget Management Page:



Reports Page (with Chart & Table):



Notification Dropdown:



Demo Link:

<https://drive.google.com/file/d/1wWzwLMrrxgJuy0nyHoObAGQ392ARej8l/view?usp=sharing>

12. Known Issues:

- Notification System: Relies on frontend polling (setInterval), which is not real-time and can be inefficient. A WebSocket or Server-Sent Events (SSE) implementation would be better. Notification persistence/clearing logic could be more robust.
- Receipt Handling: Only a text input for a URL is provided. Actual file upload, storage (e.g., S3, Cloudinary), and retrieval are not implemented.
- Business Scenario: The application is fundamentally single-user focused. Multi-tenancy, user roles (Admin/Employee), and dedicated employee expense submission workflows are not supported.
- Error Handling: While basic error handling exists, more granular frontend error messages and backend logging could be added. Some edge cases might not be fully covered.
- Category Management: Users cannot add, edit, or delete categories through the UI; the list is currently hardcoded in the backend model.

13. Future Enhancements:

- Real-time Notifications: Implement WebSockets or SSE for instant budget alerts.
- File Uploads: Integrate cloud storage (S3/Cloudinary) and backend middleware (multer) for receipt uploads.
- Multi-User Functionality: Introduce Team/Family accounts with shared data access and potentially roles/permissions.
- User-Defined Categories: Allow users to manage their own expense/income categories.
- Advanced Reporting: Add more report types (e.g., spending over time line chart, income vs. expense), more filtering options (date range, tags), and data export (CSV).
- Income Tracking: Add dedicated features for tracking income sources.
- Recurring Expenses/Incomes: Functionality to automatically create recurring transactions.
- Mobile App: Develop native or cross-platform mobile applications.
- Automated Testing: Implement unit, integration, and end-to-end tests using frameworks like Jest, React Testing Library, Supertest, and Cypress.
- UI/UX Refinements: Improve accessibility, add more visual themes, enhance mobile responsiveness further.